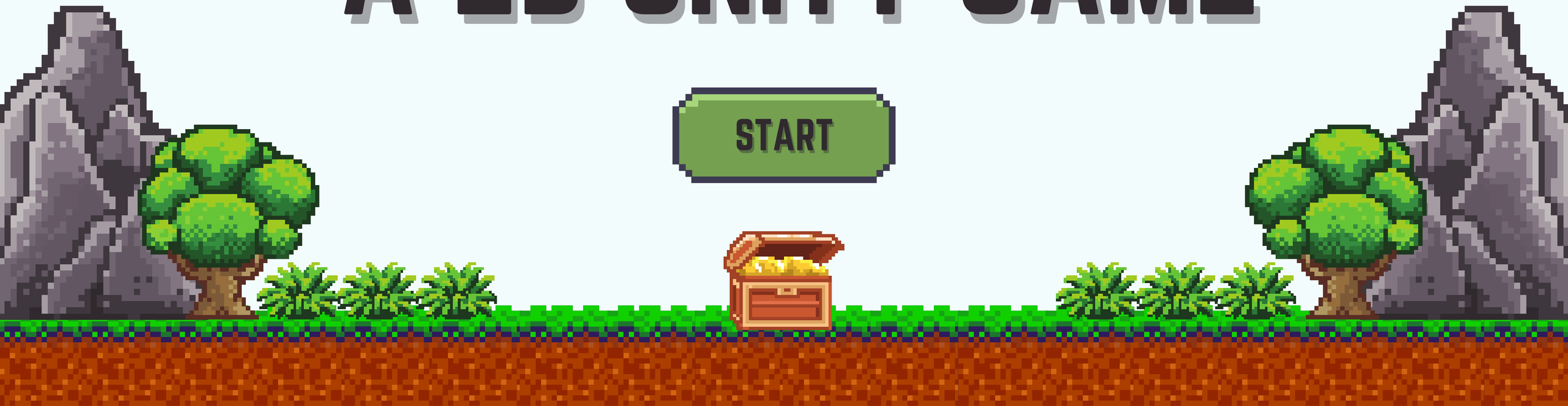
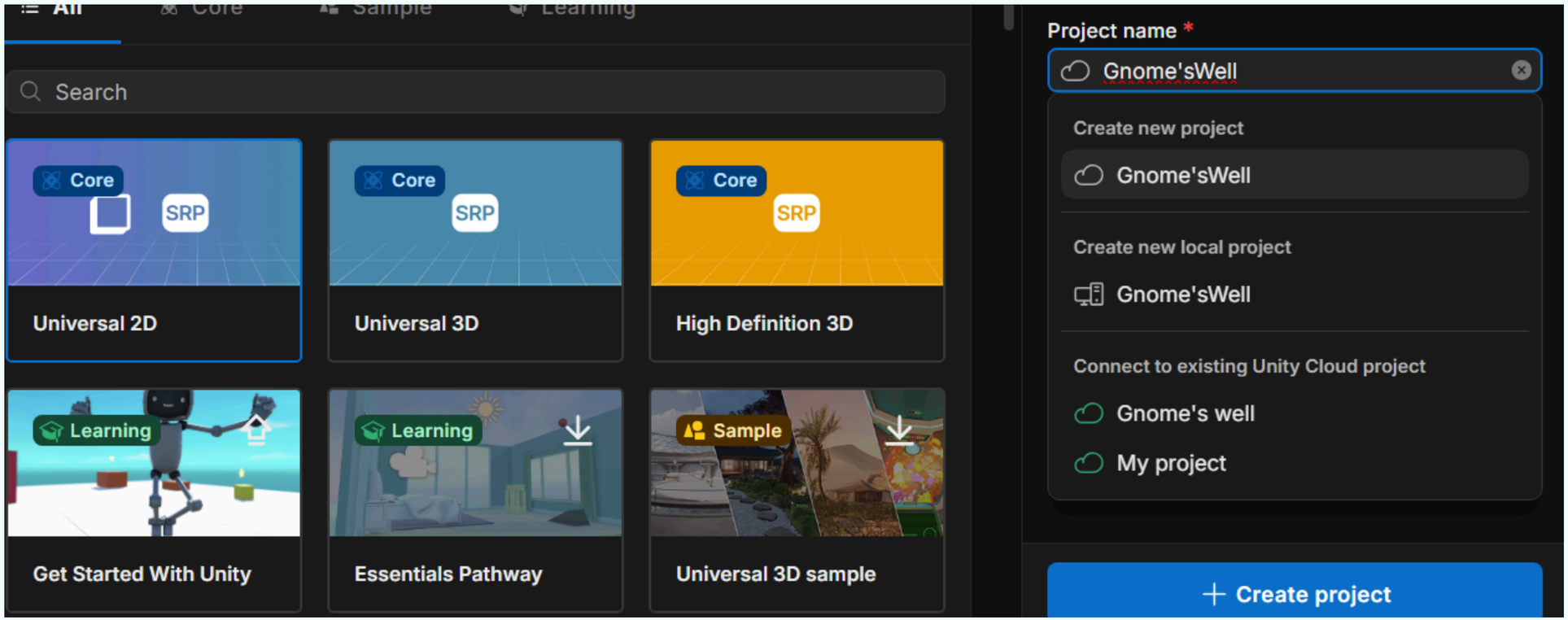
GNOME'S WELL
A 2D UNITY GAME

START



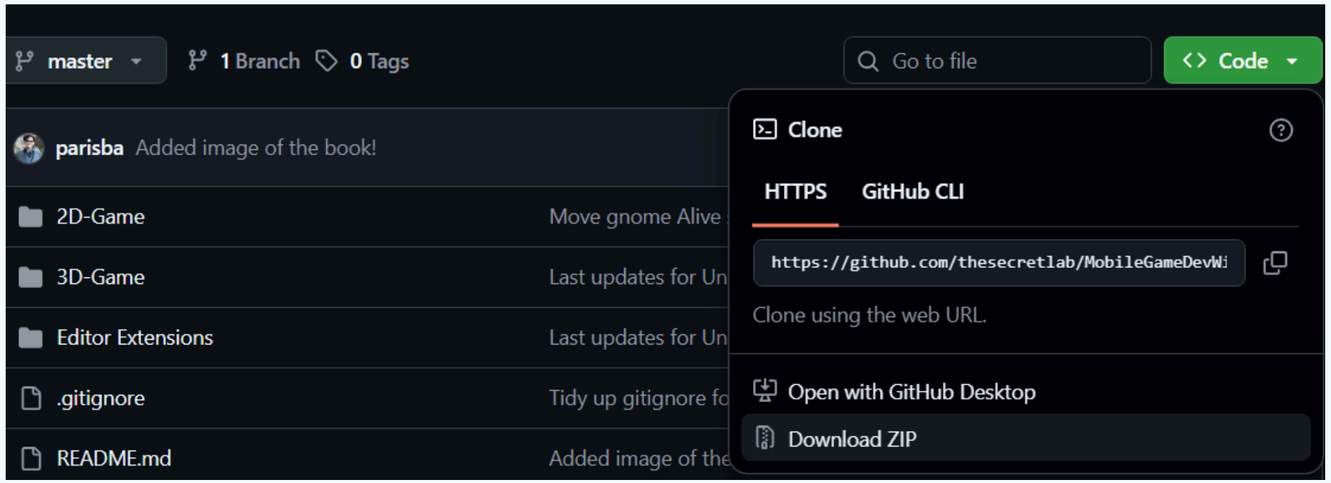
FIRST STEP

“Create a new Unity project named Gnome’s Well and set the project type to 2D to prepare for building the prototype game.”



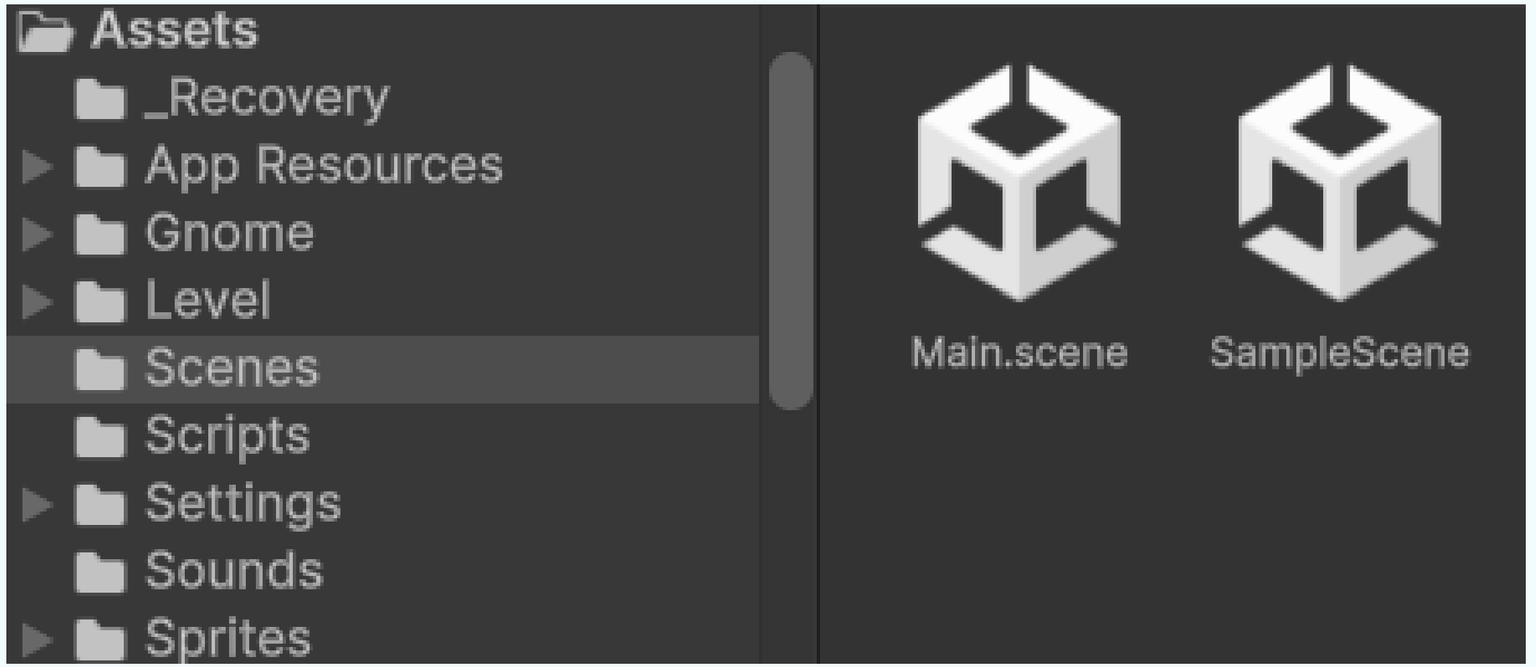
SECOND STEP

Download the required assets for the project, including art, sound, and other resources, from
“Create a new Unity project named Gnome’s Well and set the project type to 2D to prepare for building the prototype game.”
. Once downloaded, unzip the files into a dedicated folder on your computer. These assets will be imported into the Gnome’s Well project as development progresses



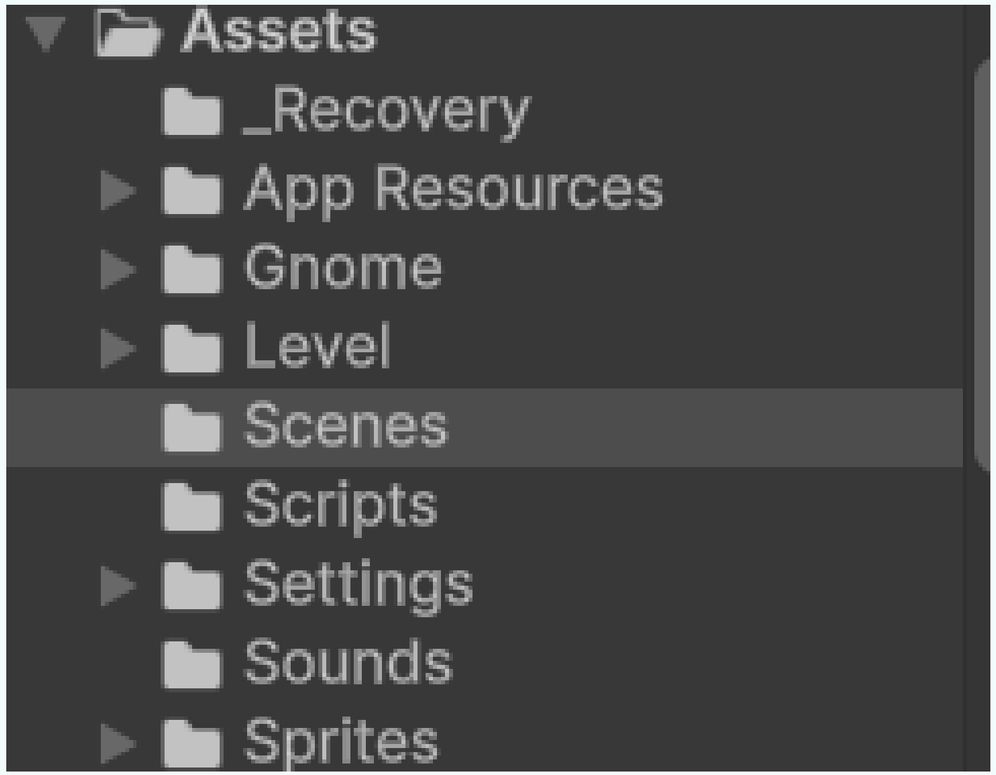
THIRD STEP

Save your scene by pressing Command + S (Mac) or Ctrl + S (Windows). The first time you save, name the scene Main.scene and store it in the Assets folder. This will allow you to quickly save and update your work as you progress.



FOURTH STEP

Inside your Unity project's Assets folder, create the following subfolders to keep your project organized: Scripts, Sounds, Sprites, Gnome, Level, and App Resources

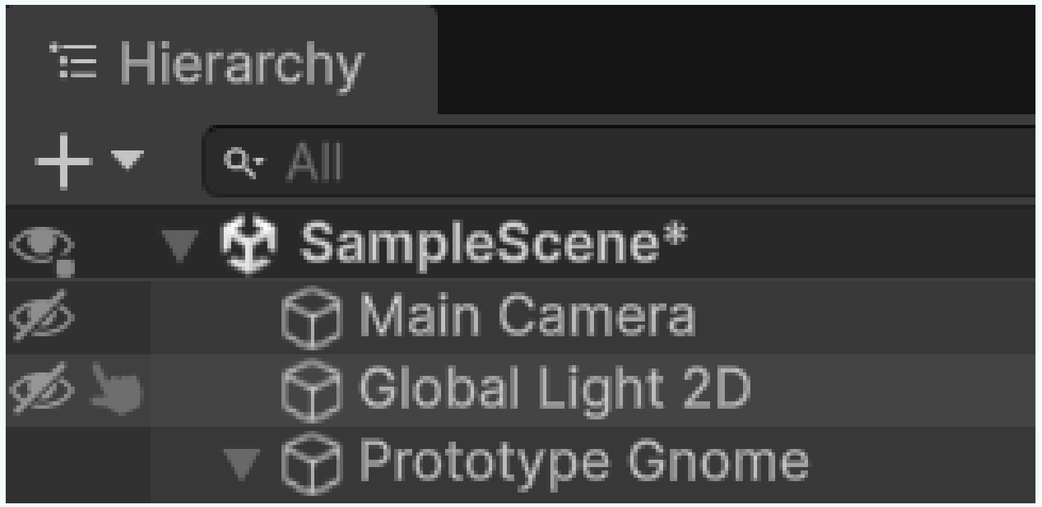


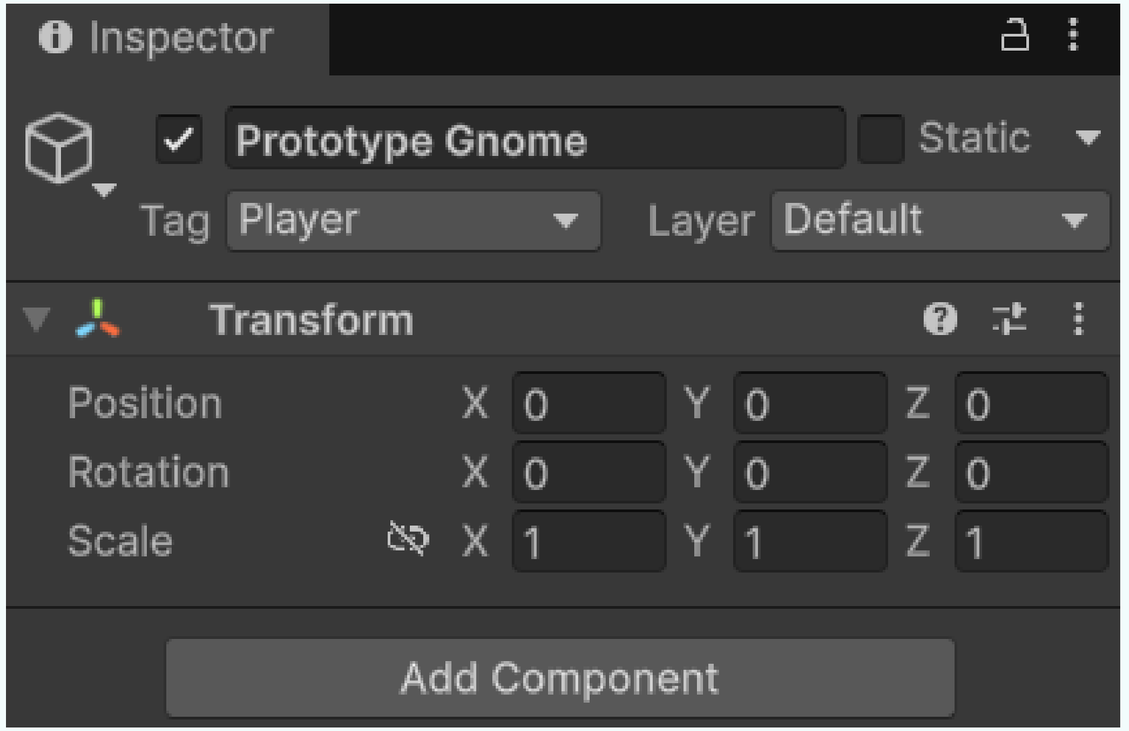
FIFTH STEP

In Unity, drag the Prototype Gnome folder from your downloaded assets and drop it into the Sprites folder within the Assets panel. This will import all the gnome sprite files into your project

CREATING THE GNOME

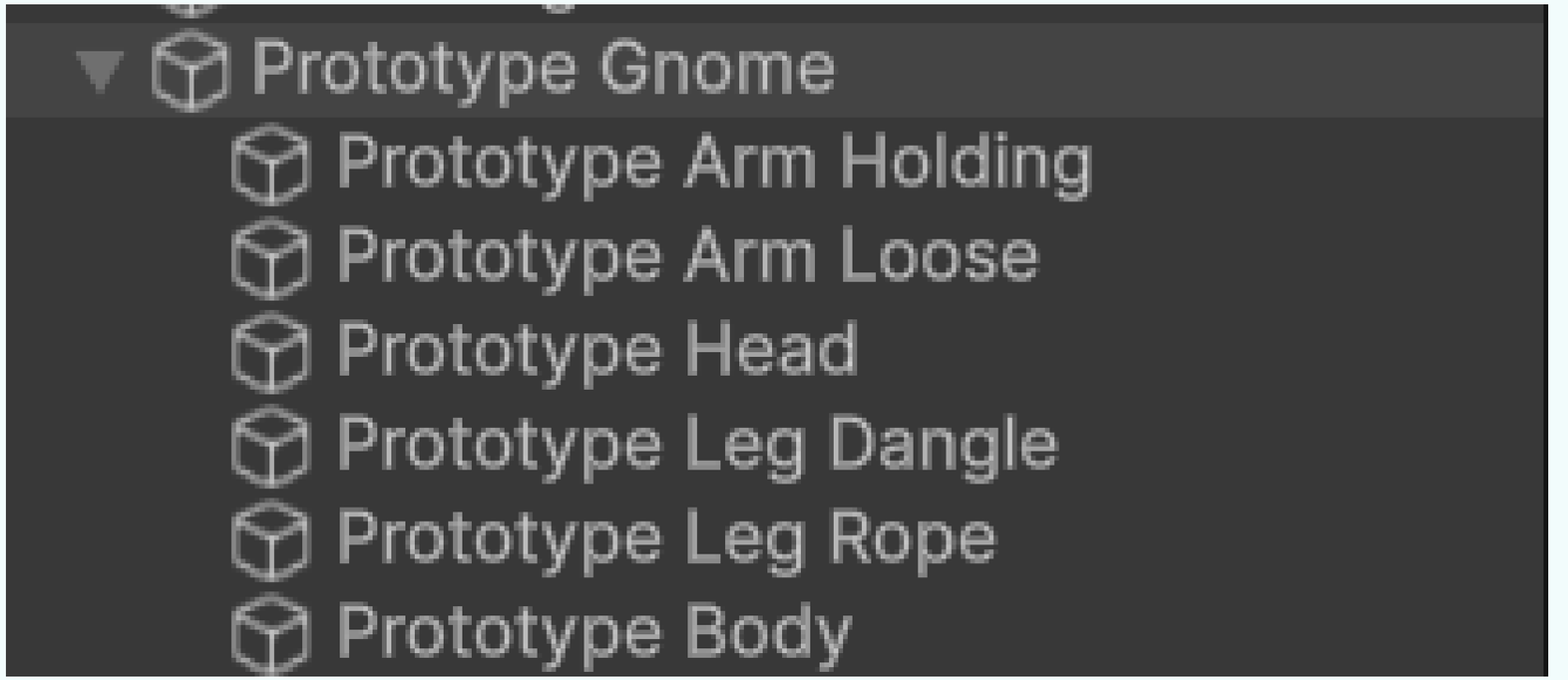
Create a new empty GameObject and rename it Prototype Gnome. In the Inspector, set its Tag to Player, then reset its Position to (0, 0, 0) to place it at the center of the scene.





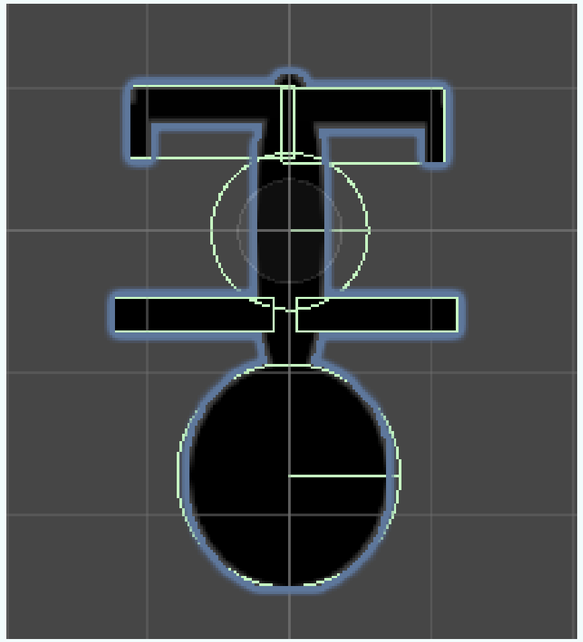
ADD THE SPRITES

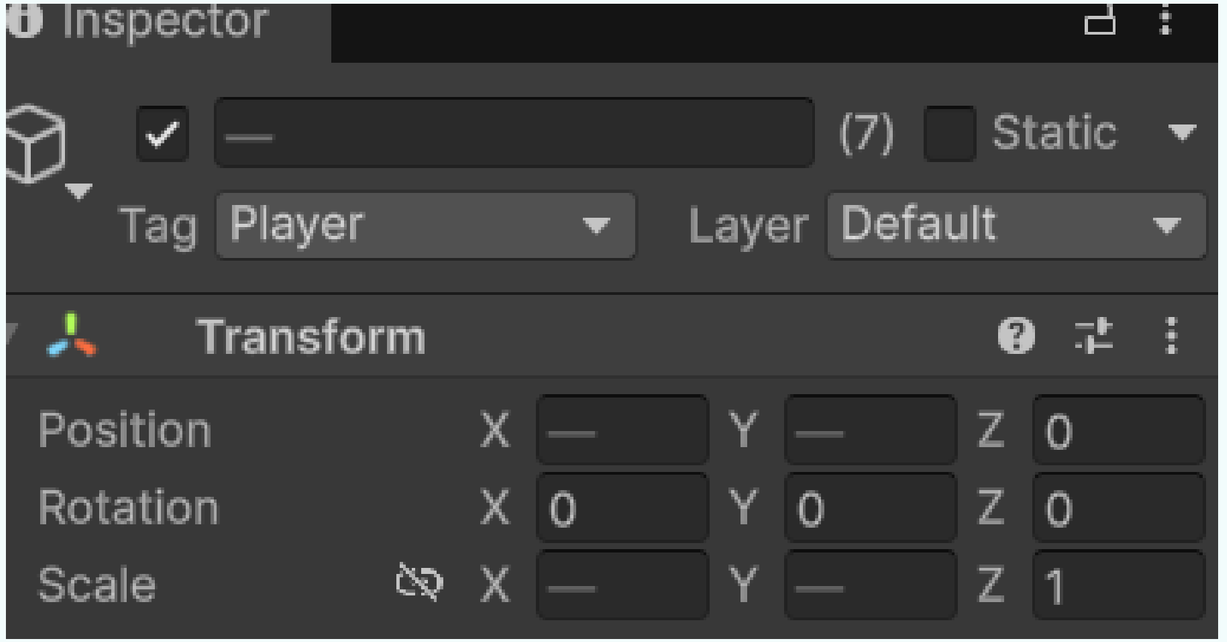
In the Hierarchy, drag all the gnome sprites you imported onto the Prototype Gnome GameObject. This will make them child objects of the main gnome object, ensuring they move together as a single unit. Your Hierarchy should now match the structure shown in the guide



POSITION THE SPRITES

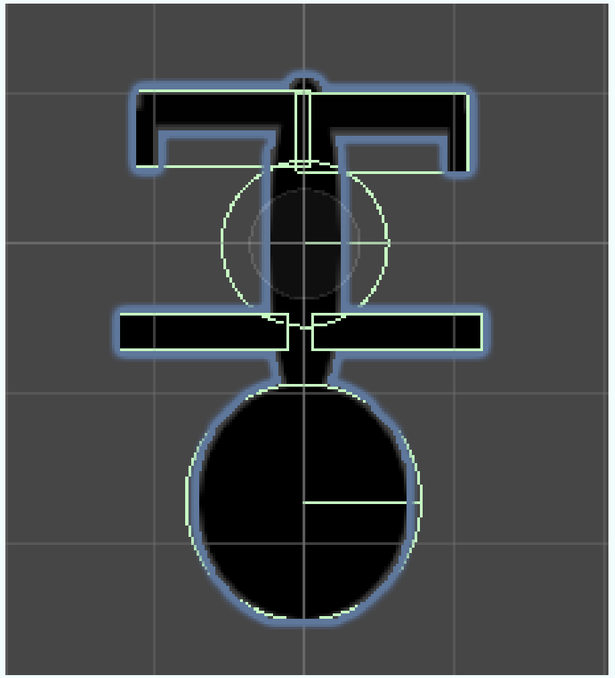
Use the Move Tool (shortcut: T) to position each sprite part (arms, legs, head) so that they form the complete gnome character. Then, for each sprite object, open the Inspector, set its Tag to Player, and ensure its Z Position is set to 0 so everything aligns correctly in 2D space





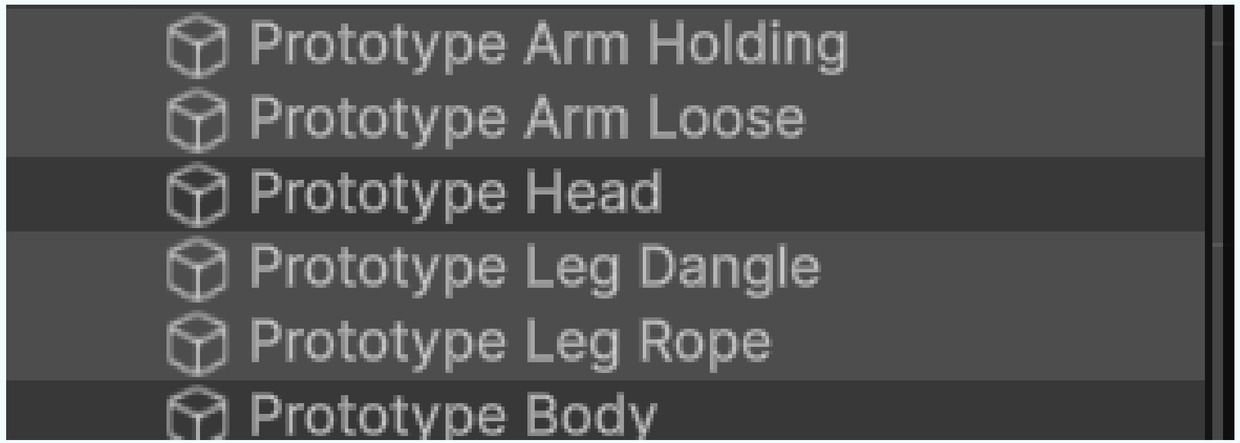
ADD RIGIDBODY 2D

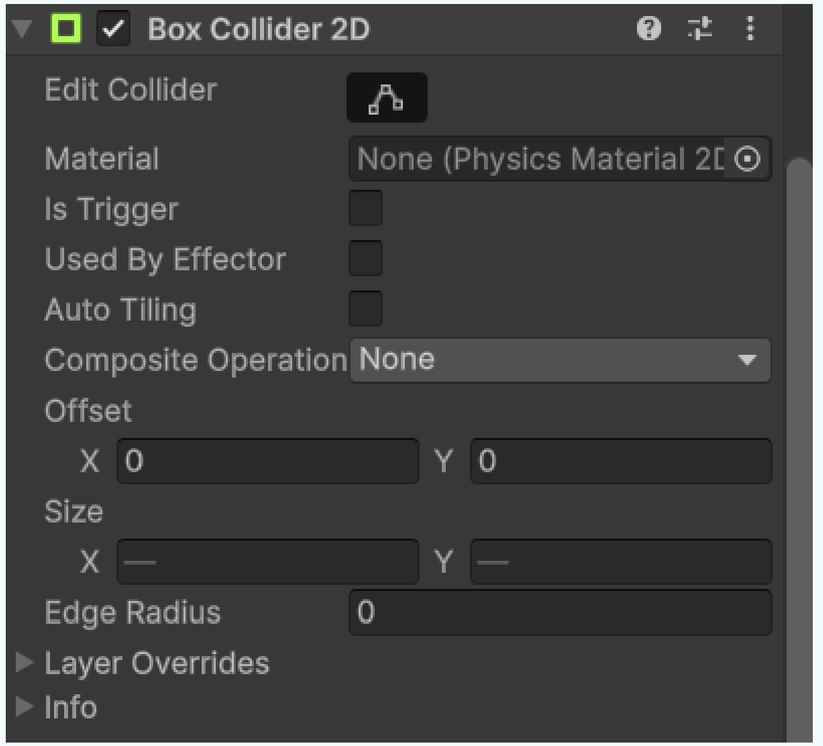
Select all the gnome body part sprites in the Hierarchy and, in the Inspector, add a Rigidbody 2D component to each one. Make sure you choose Rigidbody 2D (not the standard Rigidbody), and do not add it to the parent Prototype Gnome object—only to the individual body parts



ADD COLLIDERS

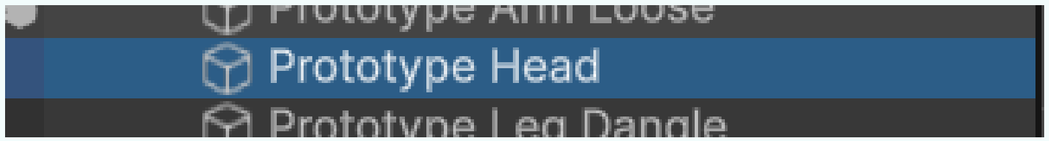
Select the arm and leg sprite objects in the Hierarchy and add a Box Collider 2D component to each one through the Inspector. This will give them physical collision boundaries for interactions in the game

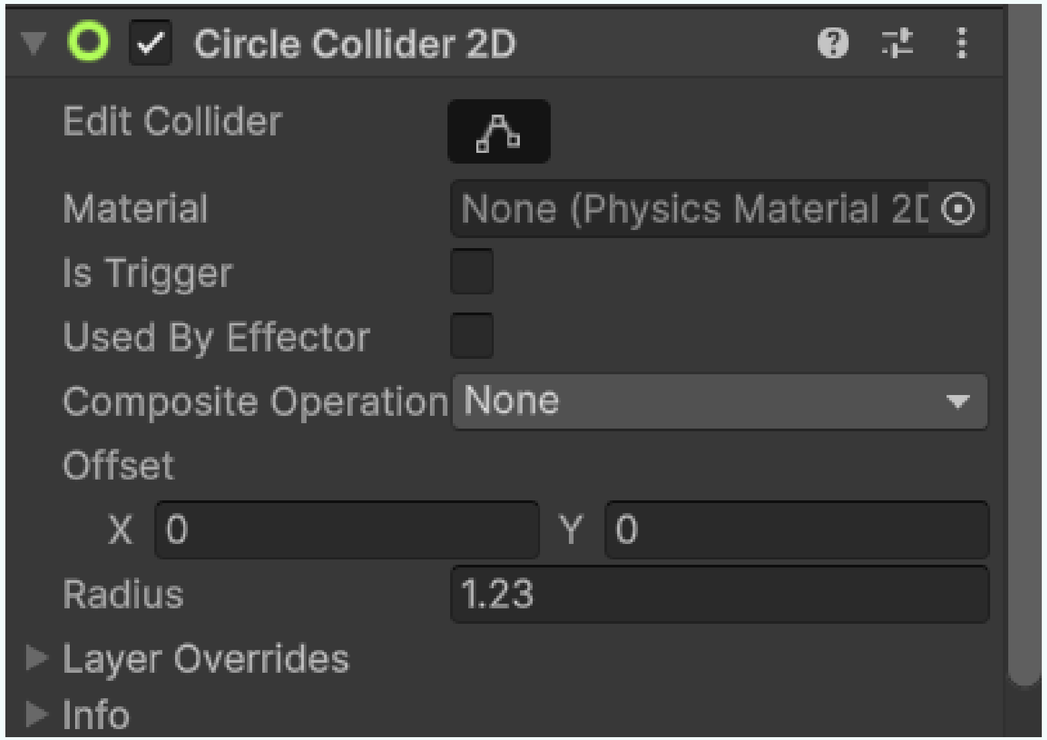




ADD COLLIDERS

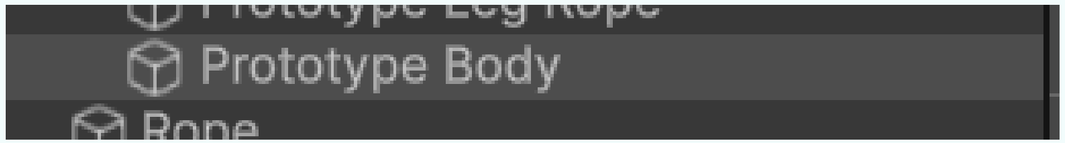
Select the head sprite in the Hierarchy and add a Circle Collider 2D component to it through the Inspector. This will give the head a circular collision boundary that better matches its shape

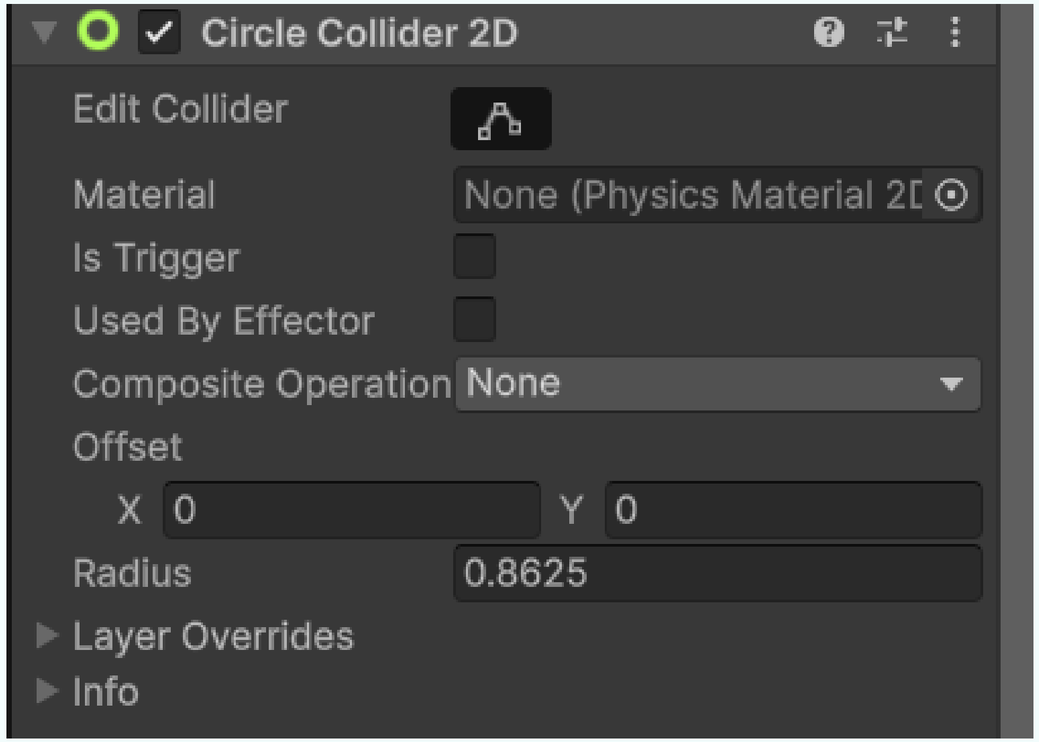




ADD COLLIDERS

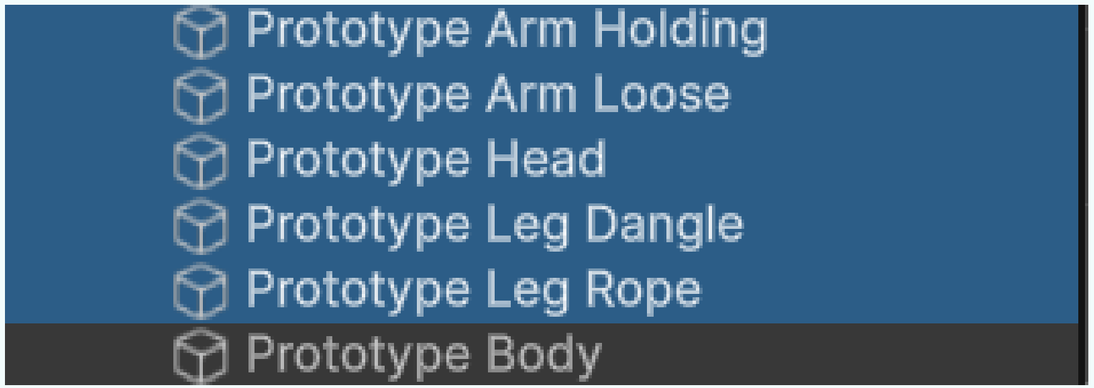
Select the body sprite in the Hierarchy and add a Circle Collider 2D component. In the Inspector, adjust the collider’s radius to about half its default size so it closely matches the body’s shape

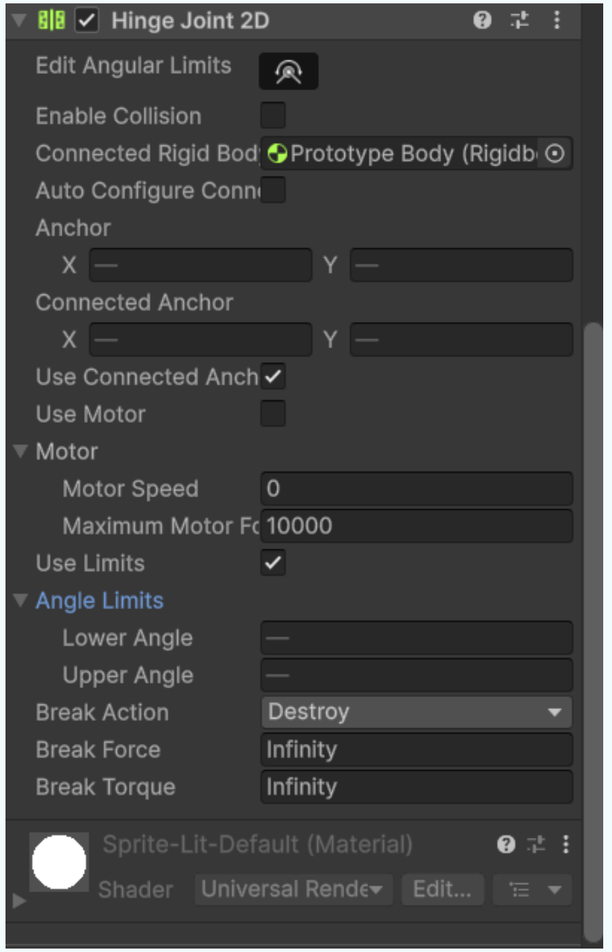




ADD HINGEJOINT2D

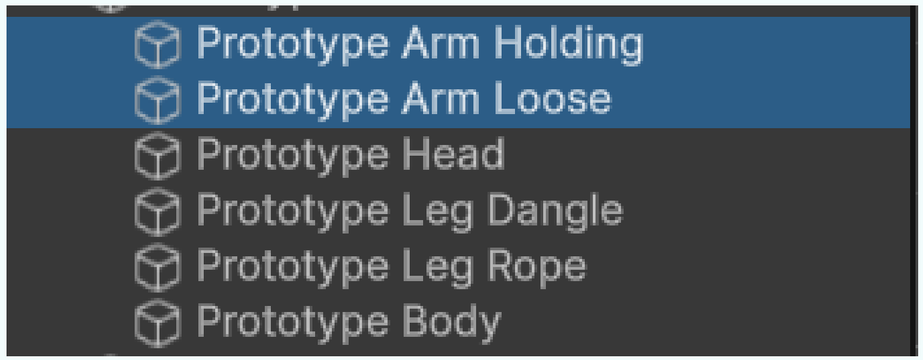
Add a Hinge Joint 2D to connect the gnome’s body parts to the main body. Select all sprites except the body, then in the Inspector, add a Hinge Joint 2D to each one. For every joint, drag the body sprite into the Connected Rigid Body field to link it to the main body





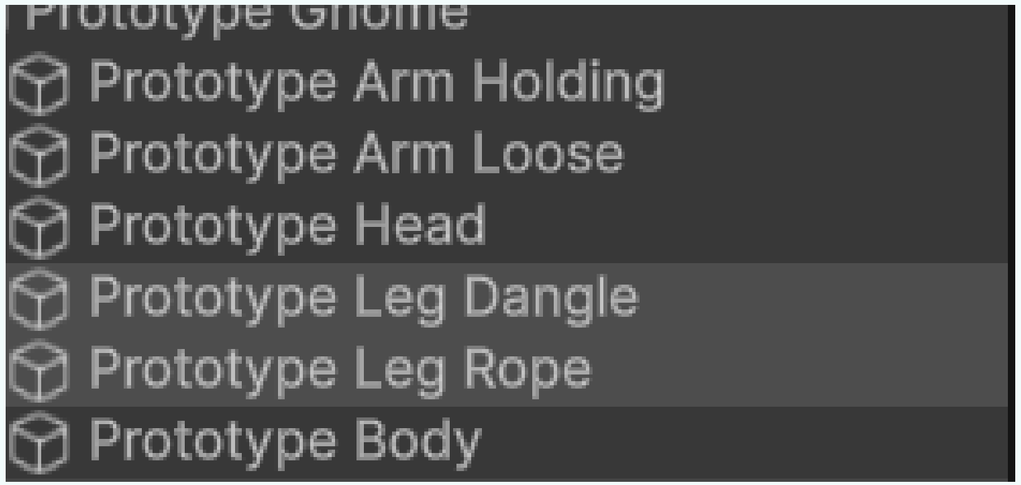
ADD HINGEJOINT2D

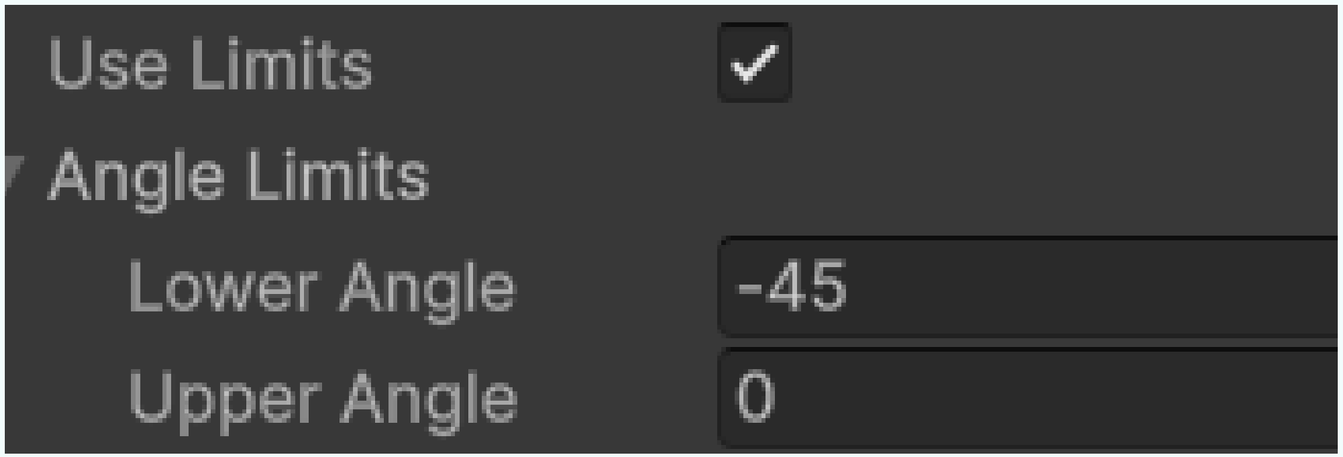
Select the arm and head sprites, then in the Inspector, enable Use Limits for their Hinge Joint 2D components. Set the Lower Angle to -15 and the Upper Angle to 15 to limit their rotation range



ADD HINGEJOINT2D

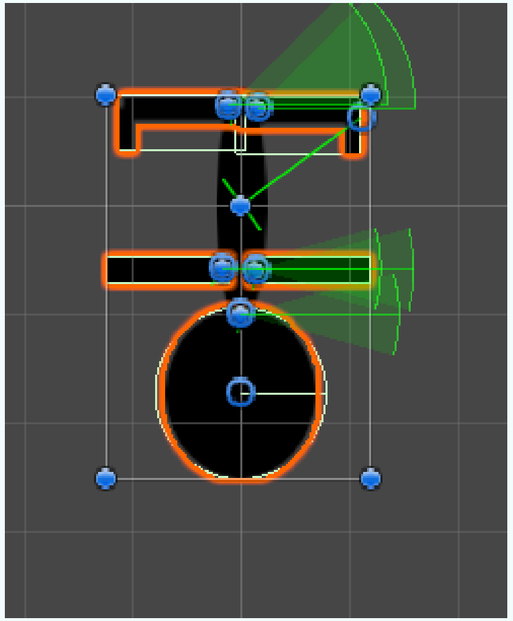
Select the leg sprites, enable Use Limits in their Hinge Joint 2D components, then set the Lower Angle to -45 and the Upper Angle to 0. This restricts the legs from rotating backward, keeping their movement natural





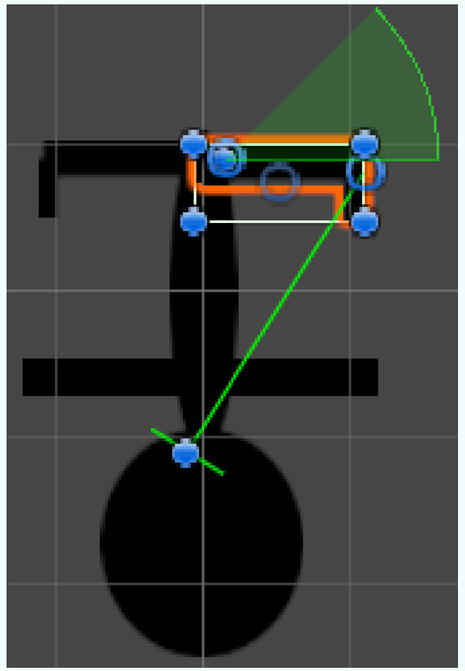
ADD HINGEJOINT2D

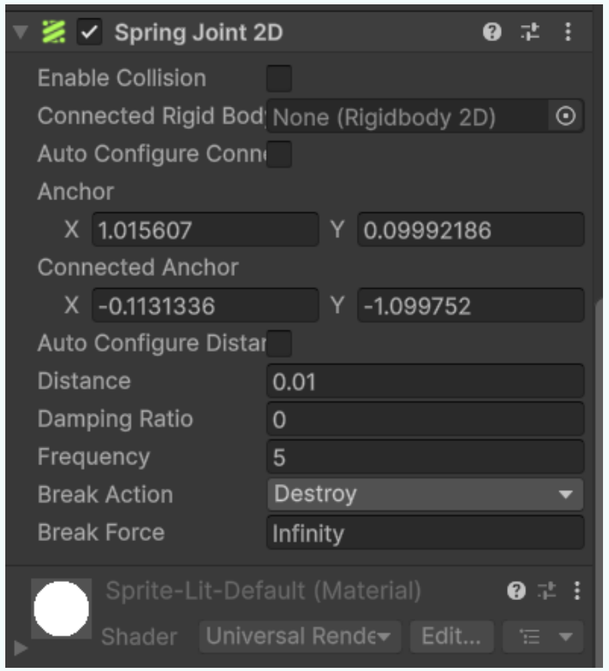
To correct the pivot points, adjust both the Anchor and Connected Anchor for each Hinge Joint 2D. Drag the Connected Anchor (blue dot) into the correct position directly in the Scene view. Then, manually edit the Anchor values in the Inspector and drag the Anchor (blue circle) to align with the Connected Anchor. Repeat this process for all joints to ensure accurate pivot points



ADD SPRING JOINT2D

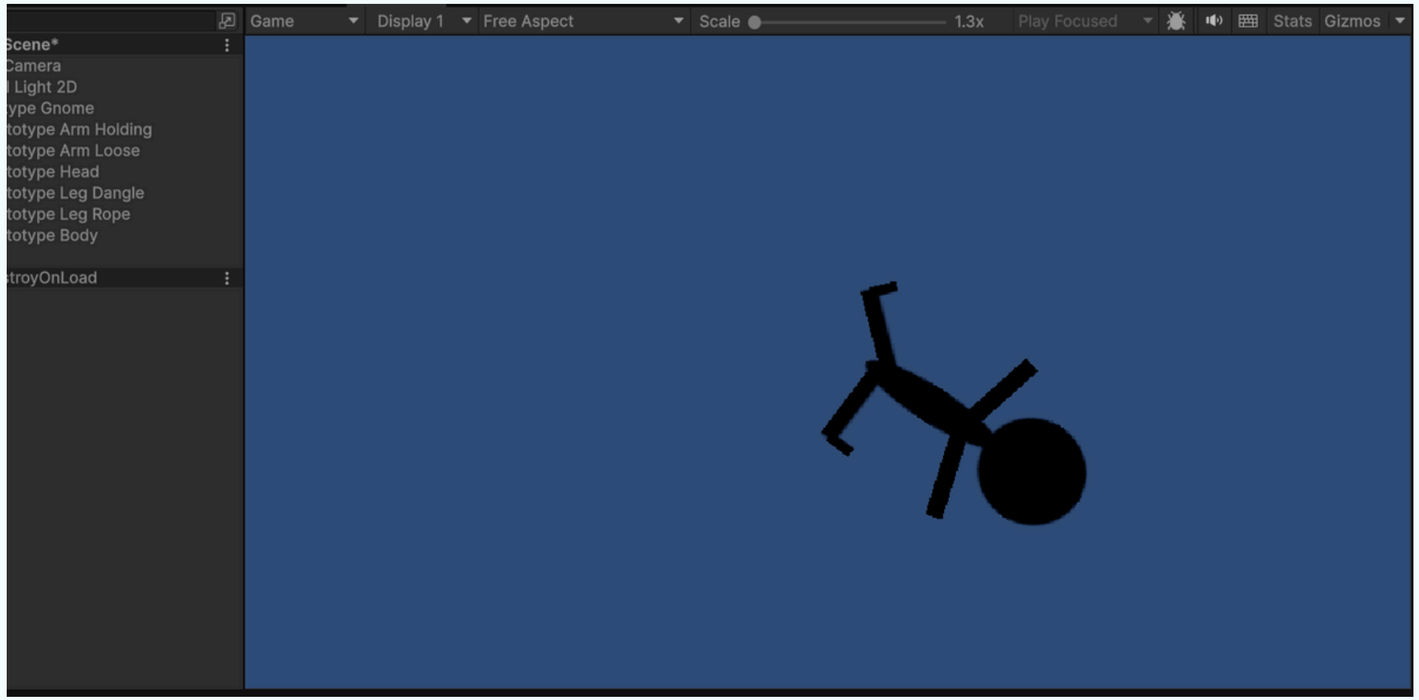
Select the Prototype Leg Rope sprite and add a Spring Joint 2D component. Move the joint's Anchor (blue circle) to the end of the leg. Then, disable Auto Configure Distance and set the Distance to 0.01 and Frequency to 5 to make the rope behave in a bouncy, stretchy way when attached





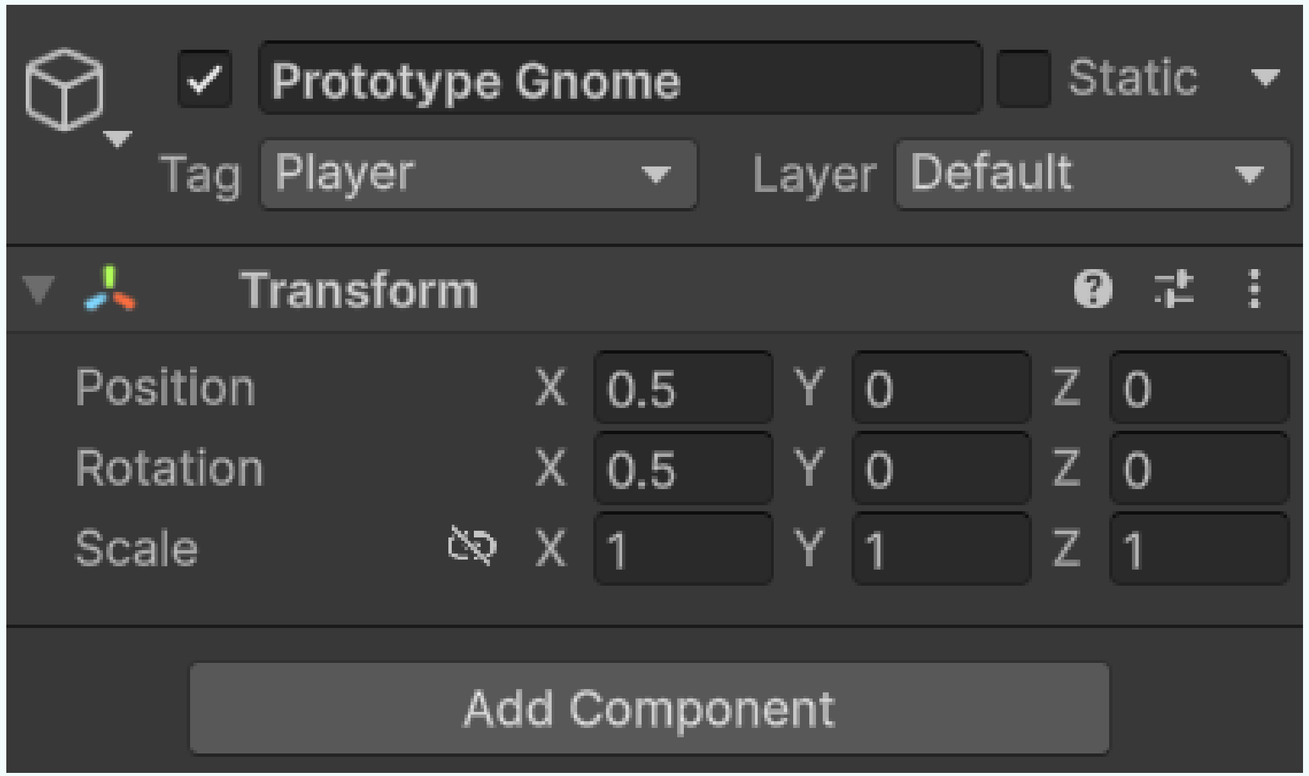
RUN THE GAME

Click the Play button at the top of the Unity Editor to test your setup. You should see the gnome’s body parts dangling and reacting to physics. Once you’ve confirmed the joints are working correctly, click Play again to stop the game



RUN THE GAME

Select the parent Prototype Gnome object in the Hierarchy. In the Transform component, set both the X Scale and Y Scale values to 0.5. This will shrink the entire gnome to half its original size

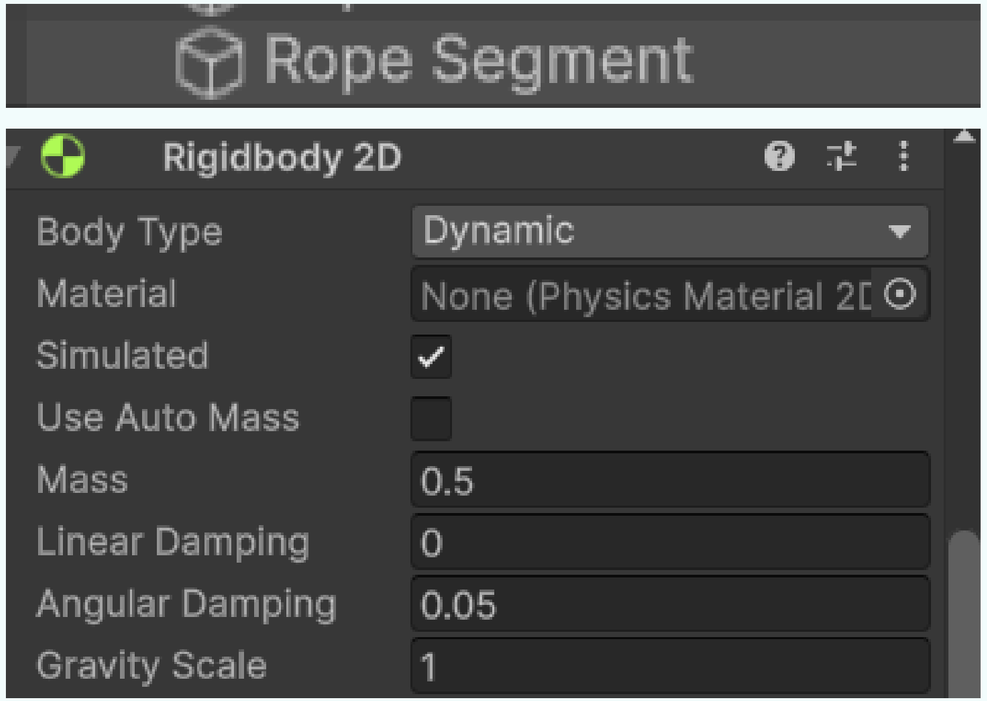


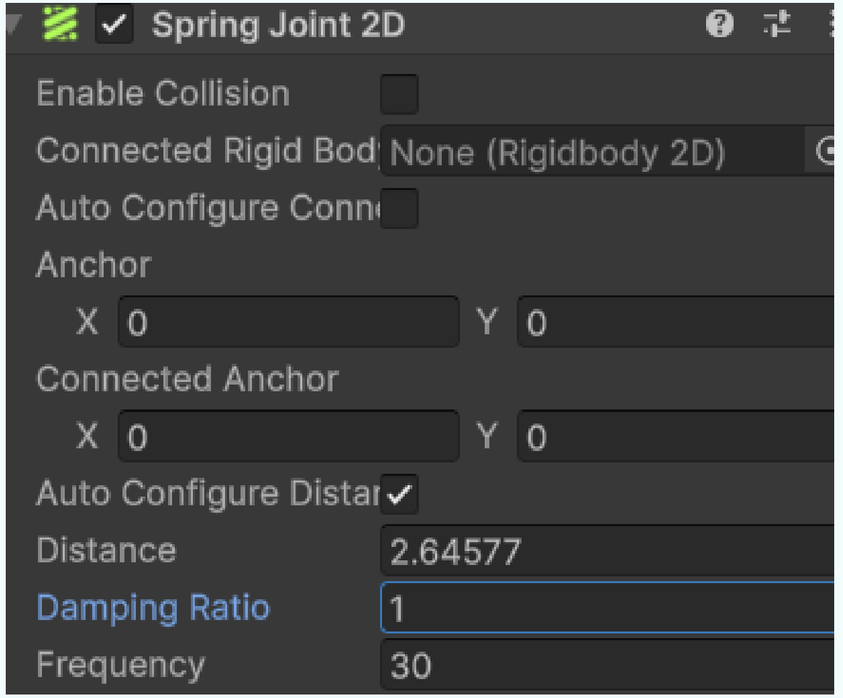
ROPE

Create a new empty GameObject and name it Rope Segment. This will serve as the base object for building the rope

ROPE

With the Rope Segment object selected, add a Rigidbody 2D component and set its Mass to 0.5. Then, add a Spring Joint 2D component, set the Damping Ratio to 1, and the Frequency to 30. These settings will give the rope a sense of weight and realistic, springy movement



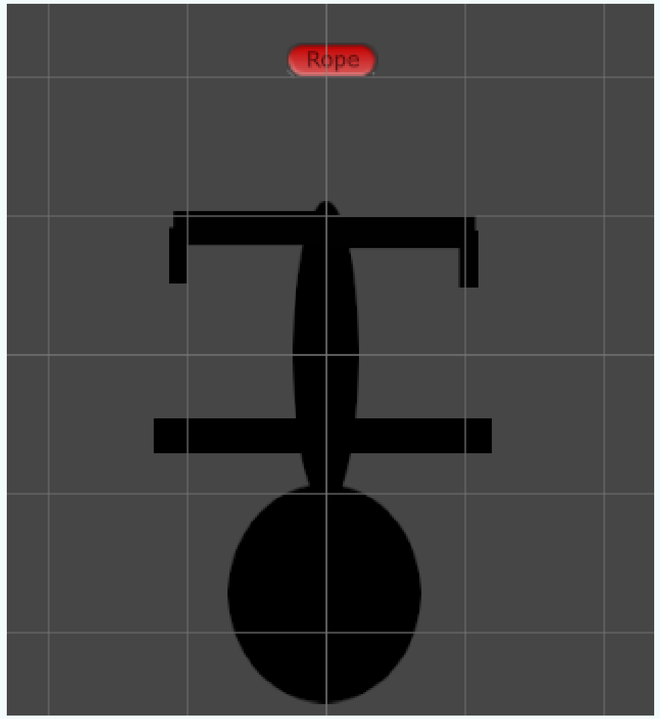


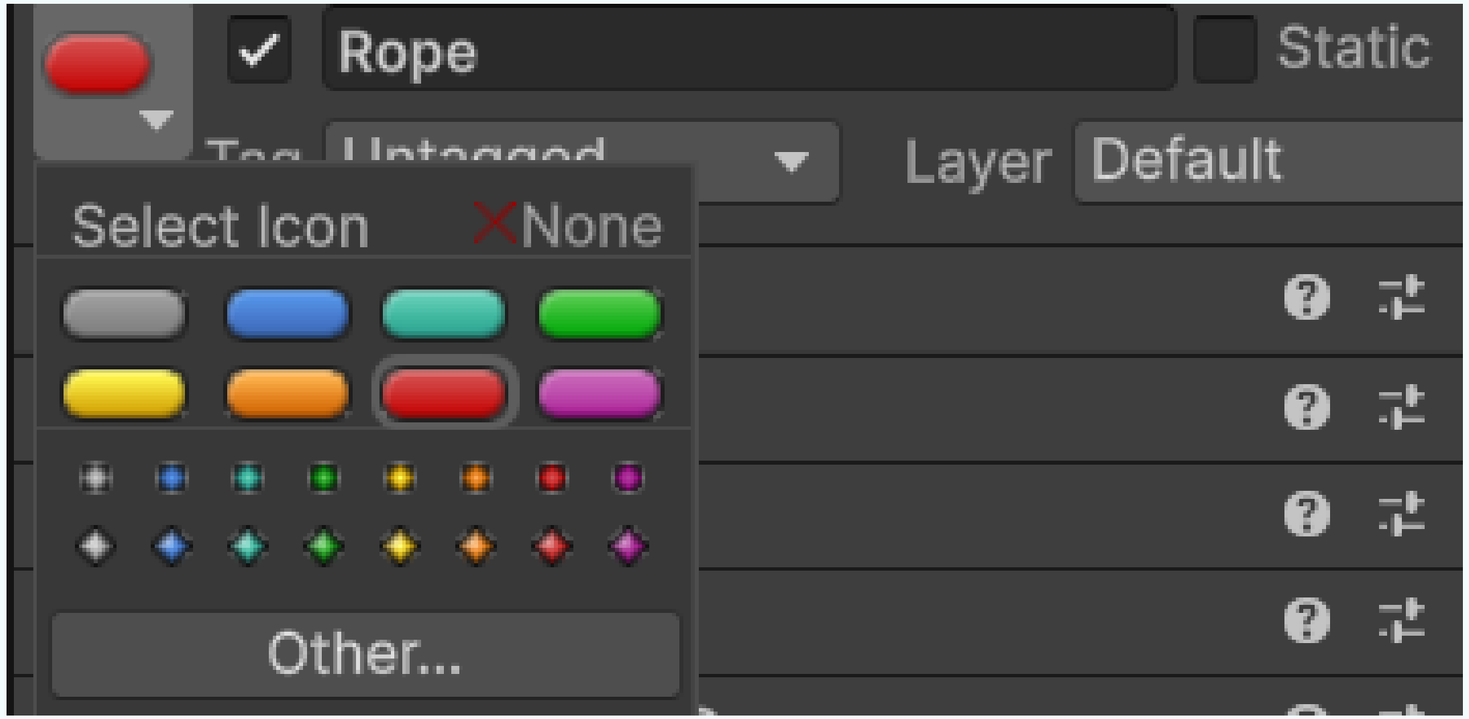
ROPE

Create a prefab by dragging the Rope Segment object from the Hierarchy into the Assets panel. Once the prefab is created, delete the original Rope Segment object from the Hierarchy. This prepares your project for writing code that will instantiate multiple rope segments during gameplay

ROPE

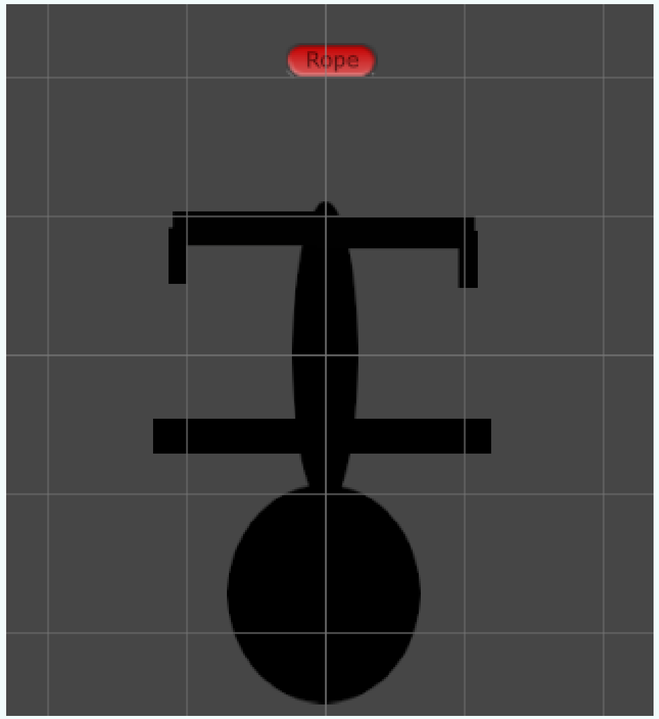
Create a new empty GameObject and name it Rope. For better visibility in the Scene view, change its Icon in the Inspector to the red rounded rectangle





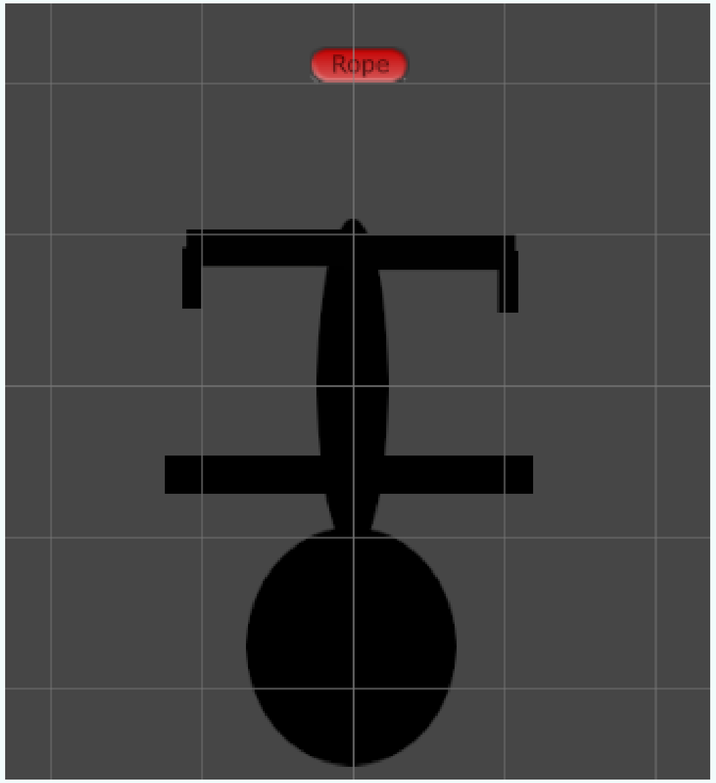
ROPE

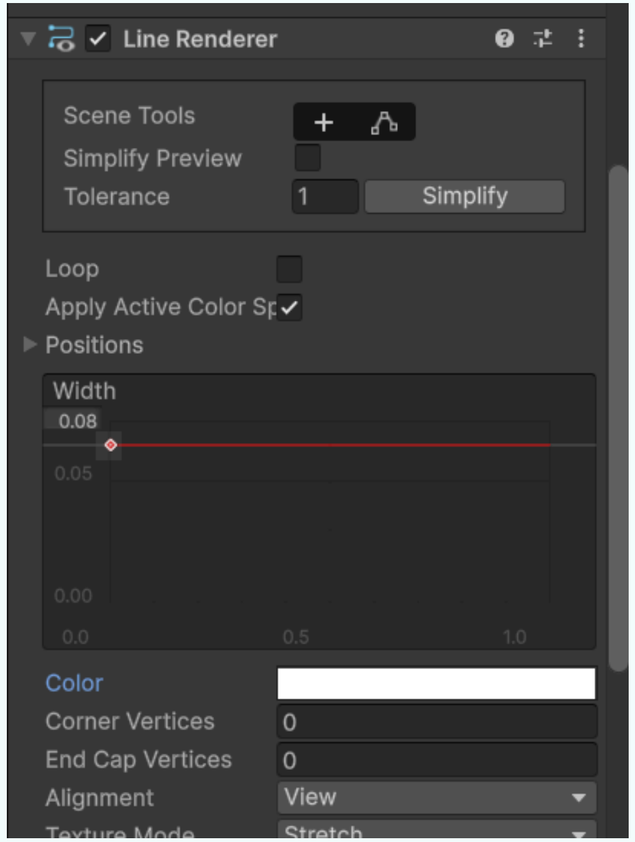
Select the Rope object, add a Rigidbody 2D component, and set its Body Type to Kinematic in the Inspector. This will keep the rope's root object fixed in place while still allowing its connected segments to move



ROPE

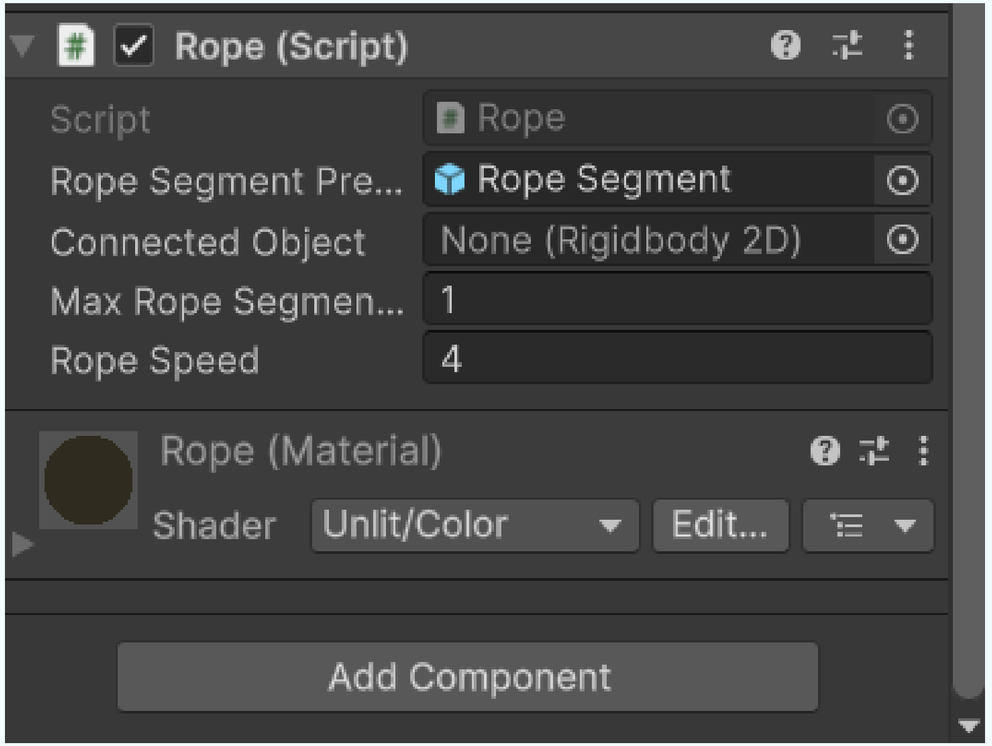
Select the Rope object and add a Line Renderer component. In the Inspector, set its Width to 0.075 to give it a thin, rope-like appearance





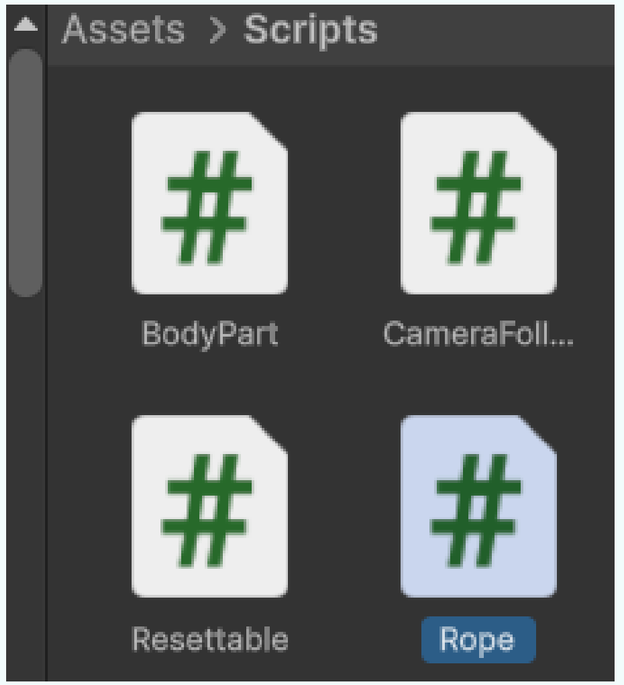
CODING THE ROPE

Select the Rope object, then click the Add Component button in the Inspector. Type "Rope" in the search bar and choose the New Script option to create and attach a new script file



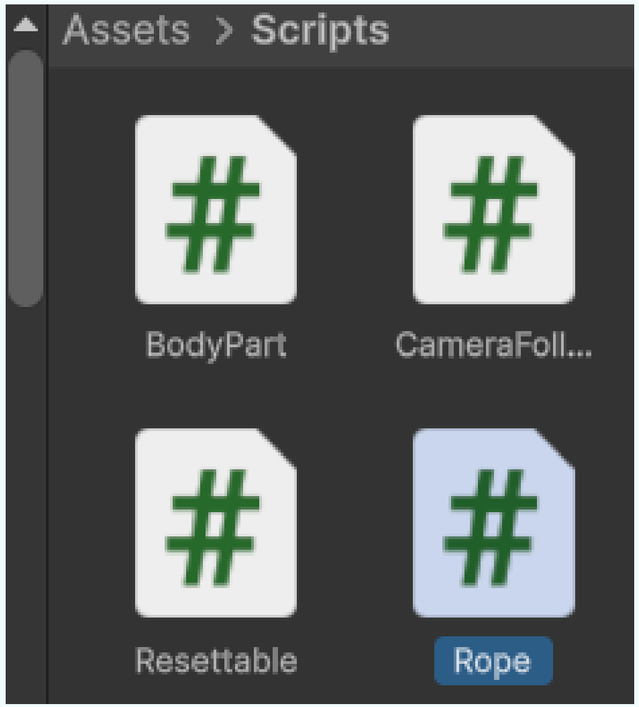
CODING THE ROPE

Make sure the language is set to C# and the script is named Rope with a capital "R." Click Create and Add to generate the Rope.cs file and attach the script to your object. To keep your project organized, drag the newly created Rope.cs file from the Assets folder and drop it into your Scripts folder



CODING THE ROPE

Open the Rope.cs file by double-clicking it, or open it in the text editor of your choice. Once the file is open, add the necessary code to define the Rope's behavior.



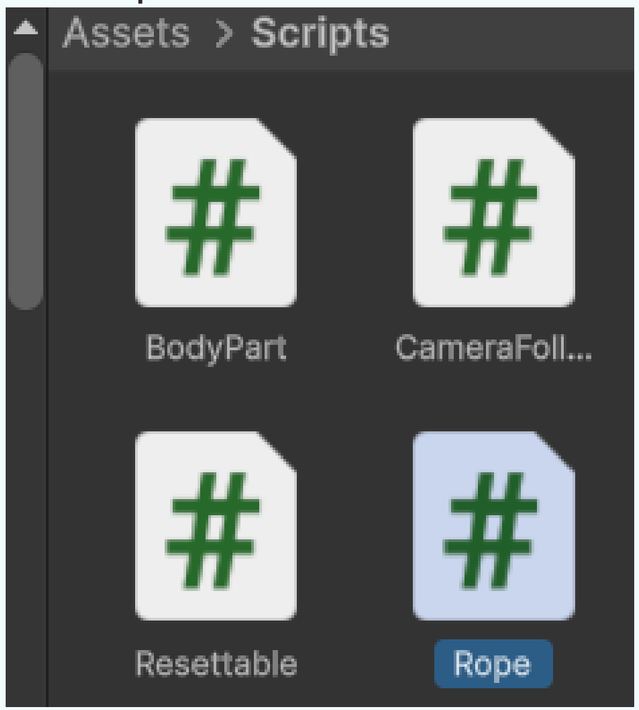
prite-Lit-Default.shader | Rope.cs | Singleton.cs

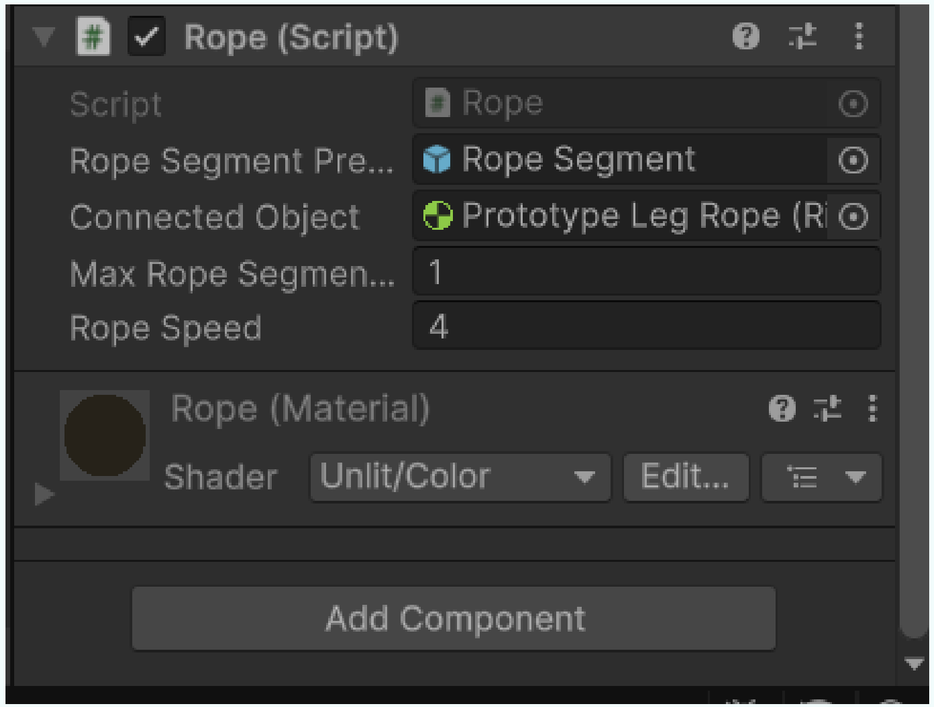
Assembly-CSharp | Rope

```
40          ropeSegments = new List<GameObject>();
41          isDecreasing = false;
42          isIncreasing = false;
43          CreateRopeSegment();
44      }
45      // Attaches a new rope segment at the top of the rope.
46      void CreateRopeSegment()
47      {
48          // Create the new rope segment.
49          GameObject segment = (GameObject)Instantiate(
50          ropeSegmentPrefab,
51          this.transform.position,
52          Quaternion.identity);
53          // Make the rope segment be a child of this object,
54          // and make it keep its world position
55          segment.transform.SetParent(this.transform, true);
56          // Get the rigidbody from the segment
57          Rigidbody2D segmentBody = segment
58          .GetComponent<Rigidbody2D>();
59          // Get the distance joint from the segment
60          SpringJoint2D segmentJoint =
61          segment.GetComponent<SpringJoint2D>();
62          // Error if the segment prefab doesn't have a
63          // rigidbody or spring joint - we need both
64          if (segmentBody == null || segmentJoint == null)
```

CONFIGURING THE ROPE

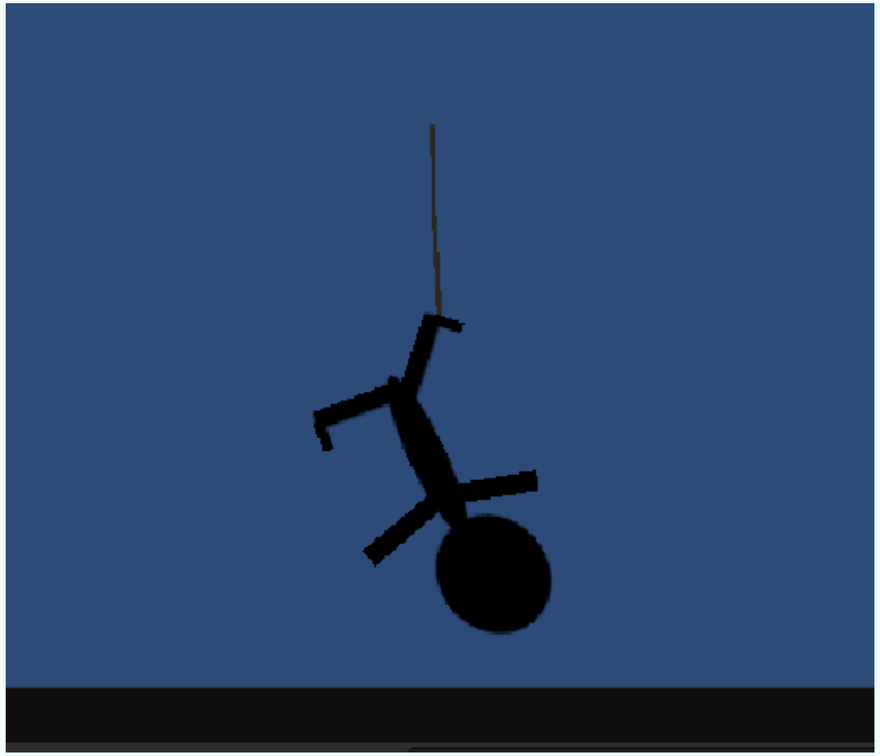
Drag the Rope Segment prefab into the Rope Segment Prefab slot, and drag the gnome's Rope Leg object into the Connected Object slot of the Rope game object. This configures the script to reference the correct components.





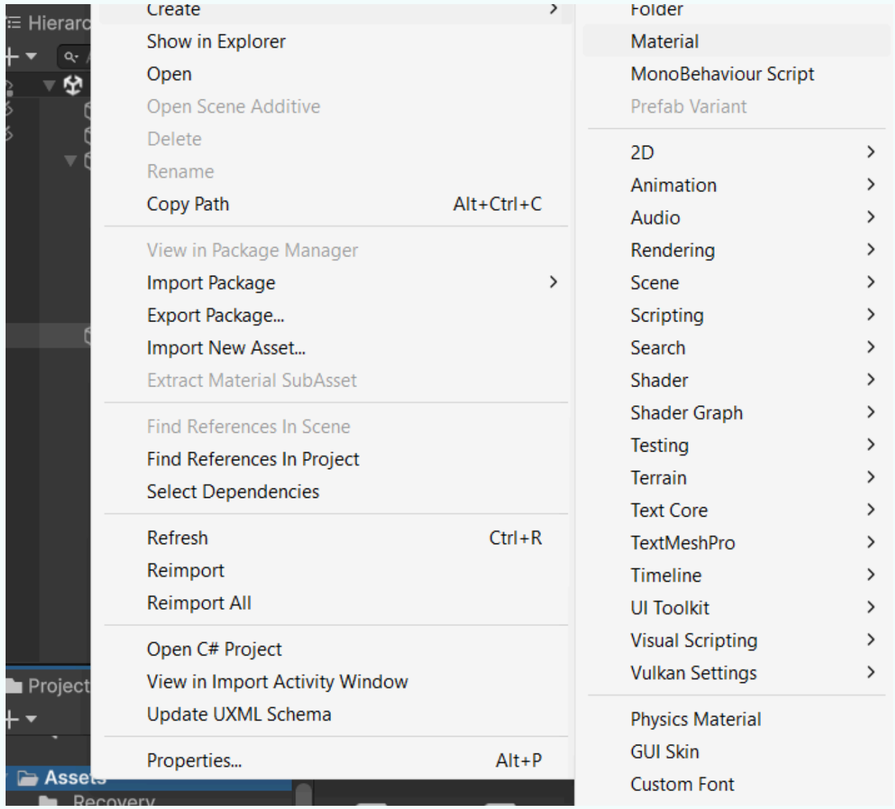
CONFIGURING THE ROPE

When you run the game, the gnome will dangle from the newly created Rope object, with a visible line connecting it to a point just above



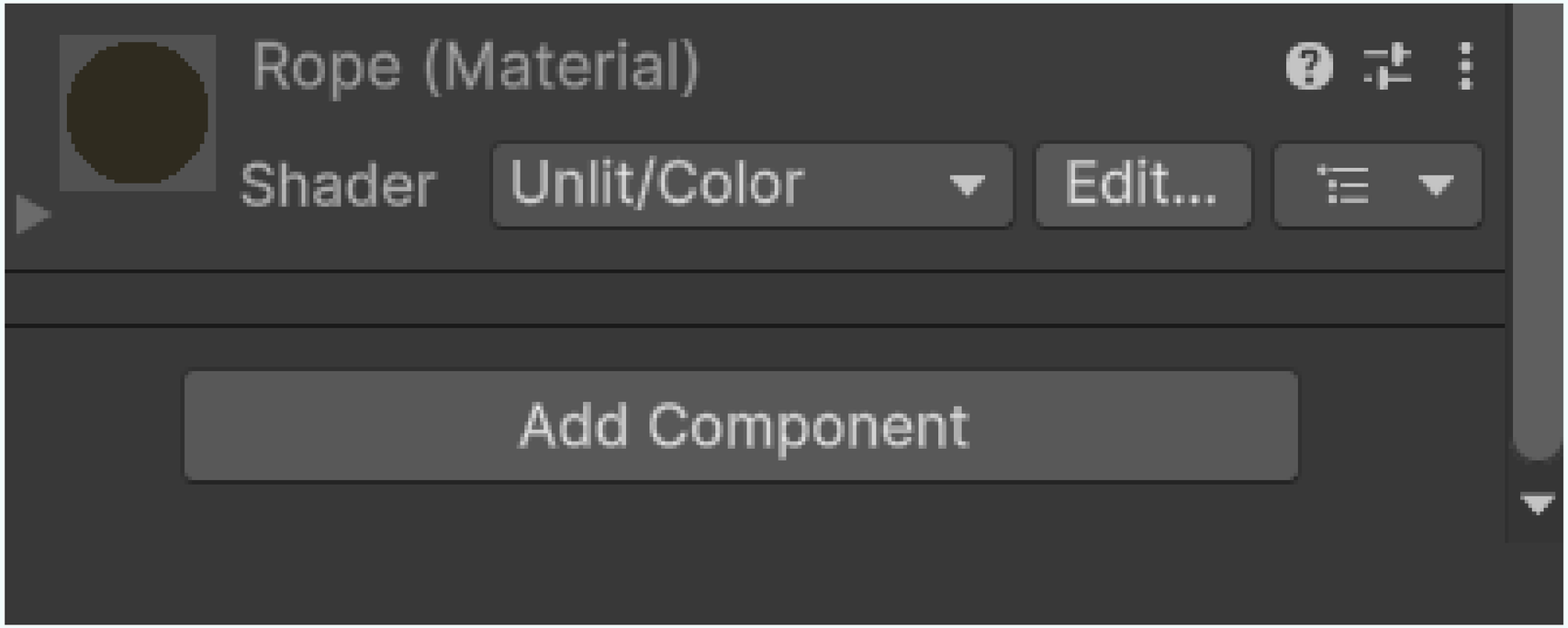
CONFIGURING THE ROPE

To create a new material, go to the Assets menu, select Create, then choose Material. Name the material Rope.



CONFIGURING THE ROPE

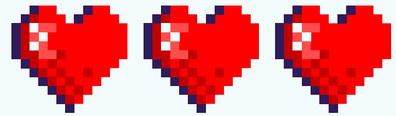
Select the new Rope material, then change its shader to Unlit → Color. Click on the color slot that appears, and in the pop-up color window, choose a dark brown color.



CONFIGURING THE ROPE

Select the Rope game object in the Hierarchy. In the Inspector, locate the Line Renderer component’s Materials property. Drag the new Rope material into the Element 0 slot.

CONCLUSION



Through this activity, you've learned how to build a basic physics system from the ground up. It’s not just about creating a rope; it’s about combining elements like SpringJoint2D and Rigidbody2D to form a fully functional object. This is a key concept in game development: using simple components to create more complex behaviors, such as lowering a gnome into a we